

P0598

To: SG7 (Reflection) & EWG

From: Daveed Vandevoorde (daveed@edg.com)

Date: 2017-02-02

Reflect Through Values Instead of Types

P0194R2 proposes adding compile-time reflection facilities to the language. I'm looking forward to the capabilities themselves, but I'm concerned about using types to perform computations when we've come such a long way in providing efficient compile-time computation facilities through `constexpr`. Let's design with modern C++ in mind and leave the template metaprogramming shackles of the past behind.

The Cost of "Types as Values"

The main downside of using types as values is that types and template instances are persistent: Every intermediate type and template instance we use in a meta-computation persists until the end of the compilation process, even though these things are only briefly needed to produce the final "value". That is almost the definition of "not scalable".

FAQ 10.3 in P0385R1 argues that the meta types can be very "lightweight" and need not be "regular" types. That doesn't address the issue of persistence (which is not just about the meta-information types, but all the templates instantiated from them), and it potentially introduces new problems: We now get a new kind of beast in our type system, which may be "heavyweight" when it comes to specification and implementation.

An Alternative Approach

Instead of trafficking in types, let's deal with ordinary values as we typically do in imperative programming. We can add a single "magical" literal class type to encapsulate a hook to compiler internals (e.g., `std::metainfo`). Along with that type, a few operators can transition back and forth between the meta and non-meta worlds. E.g.:

```
std::metainfo tp = reflexpr(X);
typename(tp) x; // Same as "X x;"
```

Instead of the `typename(...)` operator, we could use the suggested `unreflexpr(...)` operator. I find the former clearer however and it avoids a parsing problem in some contexts. Similarly, I'd suggest a `template(...)` operator for materializing templates, and maybe notations to materialize various names and operators (e.g., `x.(info)`, `N::(info)`, `(:info)(x, y)`, etc.).

We can add a literal dynamic container type (e.g., `std::constexpr_vector<T>`; it's a literal class type by fiat) to ease dealing with collections of metadata (and, incidentally, such a facility would be more broadly useful; see P0597). Beyond that, many of the facilities proposed in P00194R2 can just be translated to use ordinary function interfaces (instead of template instantiation interfaces). E.g.:

```
struct foo { ... };
typedef reflexpr(foo) meta_foo;
meta::get_public_data_members_m<meta_foo> meta_data_mems;
```

becomes

```
struct foo { ... };
constexpr auto meta_foo = reflexpr(foo);
auto meta_data_mems = get_public_data_members(meta_foo);
```

The type of `meta_data_mems` in this example could be `std::constexpr_vector<std::metainfo>` .

This approach addresses the scalability of reflection itself. It doesn't address the scalability of synthesis, since there is no `constexpr` mechanism to do that at this time (i.e., that still requires recursive template instantiations). There is, however, nothing to stop us from adding such capabilities: `idreflexpr` is an example of this, but a more general code injection system could be provided too (N1471, "Reflexive Metaprogramming in C++" has some ideas that could be adapted to the `constexpr` world).